15. AWS SDK, CLI

1. SDK

SDK: Software Development Kit(소프트웨어 개발 키트)

- 애플리케이션을 만들기 위한 소프트웨어 개발 도구의 집합
- 만드는 애플리케이션에 AWS를 연동하기 위한 개발 도구의 집합

---

2. CLI

CLI: Command Line Interface(명령행 도구)

- 터미널을 통해 명령을 실행할 수 있게 해주는 도구

---

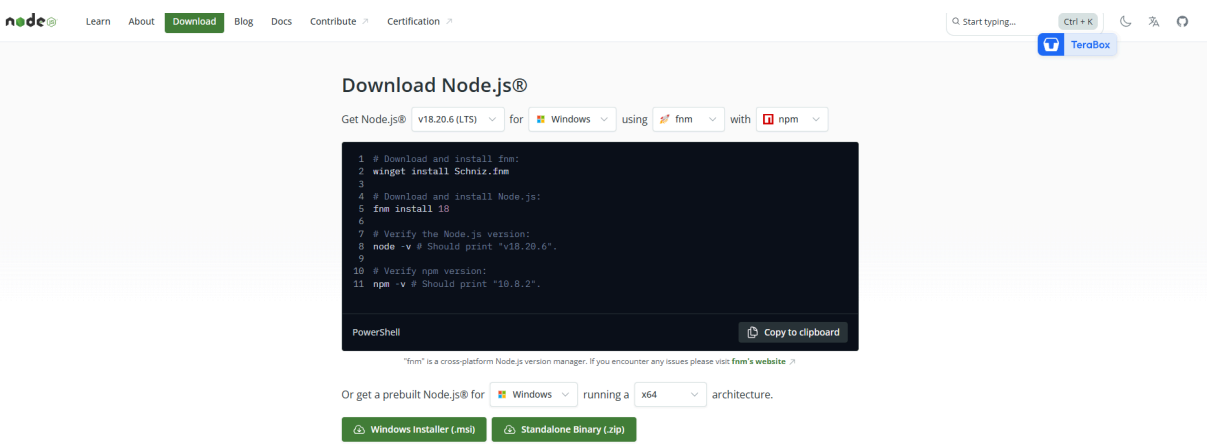
3. Node.js와 npm 설치

Node.js: 자바스크립트로 네트워크 애플리케이션을 개발할 수 있게 해주는 환경

npm(node package manager: Node.js를 위한 패키지 매니저

- 패키지 매니저: 프로젝트에서 필요로 하는 다양한 외부 패키지들의 버전과 의존성을 관리하고 편하게 설치하고 삭제할 수 있게 도와주는 역할을 한다.
- npm은 Node.js를 설치하면 자동으로 함께 설치

Node.js 설치: https://nodejs.org/en/download/



- npm 버전: 6.14.0 이상

- Node.js 버전: 최소 14.0.0 이상

```
PS C:\Windows\system32> node --version
v18.20.6
PS C:\Windows\system32> npm --version
10.8.2
```

---

4. Shared Credentials 설정

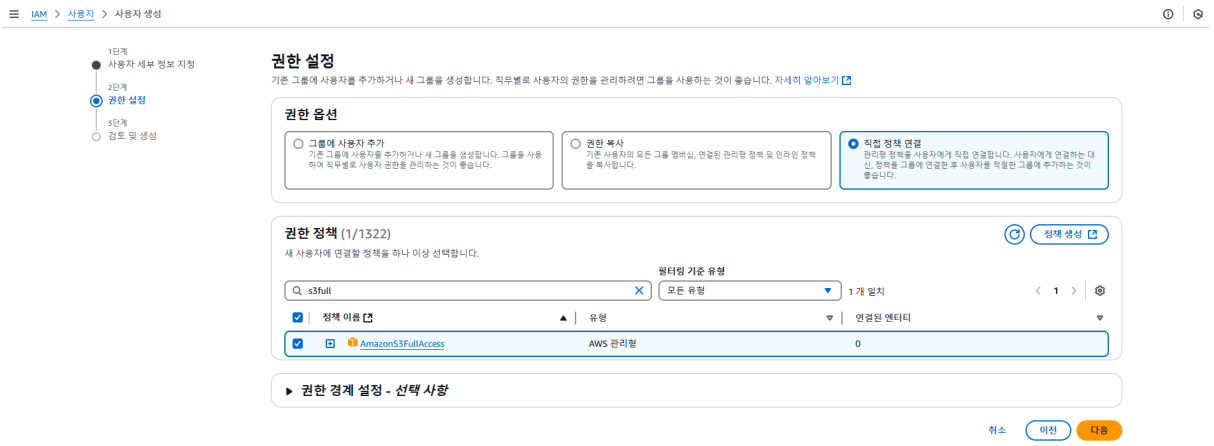
Shared Credentials(공유 자격 증명)

- 사용하는 컴퓨터에 AWS 자격 증명을 설정함으로써, 해당 컴퓨터를 사용하는 사람은 별도의 설정 없이 SDK나 CLI를 사용하여 AWS 서비스를 사용할 수 있다.
- 공유 자격 증명은 AWS를 사용해서 서비스를 개발하는 경우에 거의 필수적으로 설정해야 한다.

사용자 생성

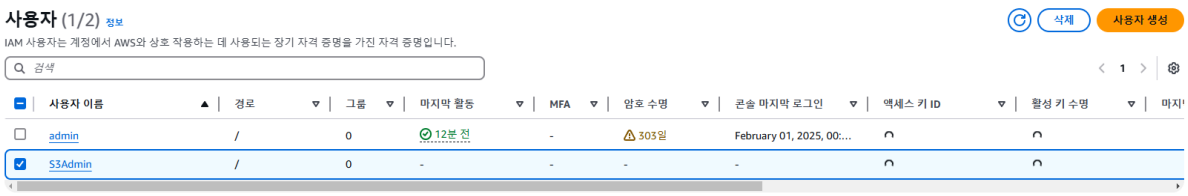
- IAM > 사용자 > 사용자 생성
- **1단계: 사용자 세부 정보 지정**
  - 사용자 이름: S3Admin

- **2단계: 권한 설정**
  - 권한 옵션: 직접 정책 연결
  - 권한 정책: s3full

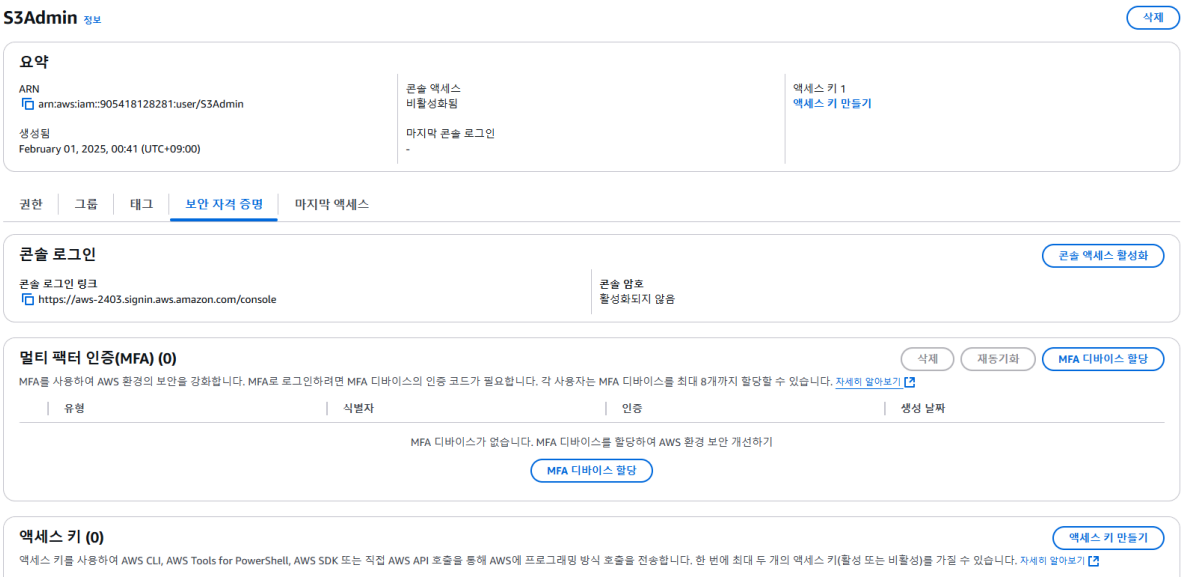


- **3단계: 검토**

- **사용자 확인**



- **보안 자격 증명 > 액세스 키 만들기**

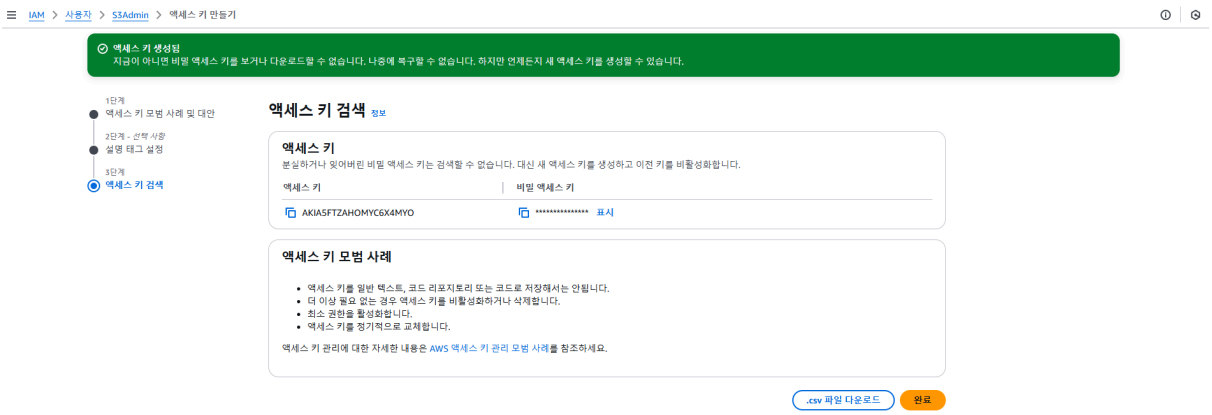


- **1단계 액세스 키 모범 사례 및 대안**
  - 사용 사례: CLI
  - 확인: 위의 권장 사항을 이해했으며 액세스 키 생성을 계속하려고 합니다. - 체크

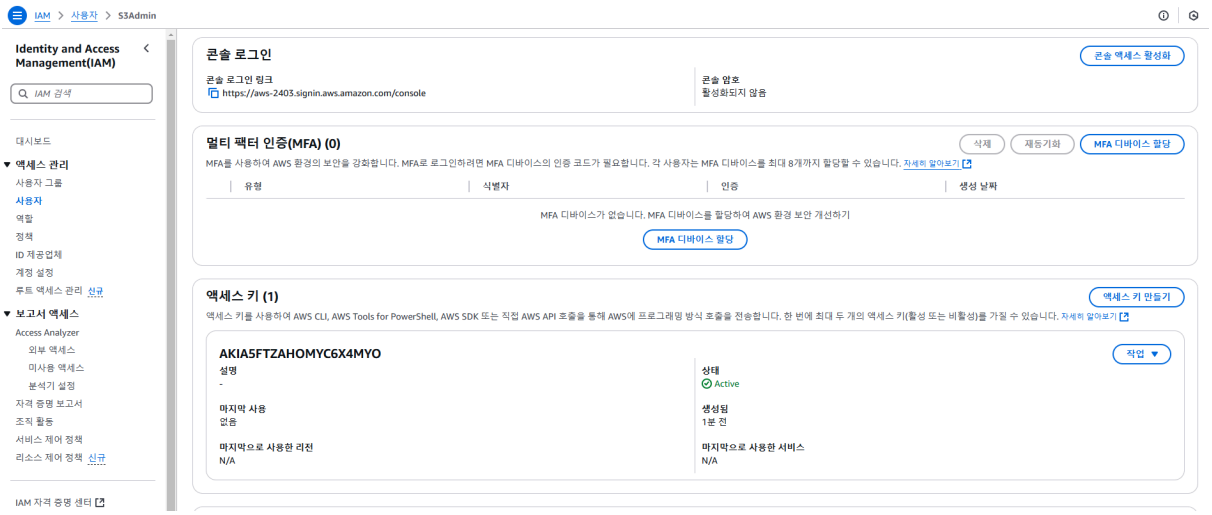
- **2단계: 설명 태그 설정 - 그대로**

- **3단계: 액세스 키 검색 - .csv 파일 다운로드**

- **지금만 다운로드 받을 수 있다.**



- **IAM 사용자의 액세스 키 목록: 액세스 키 생성 확인**



Windows의 경우엔 사전에 정해진 위치에 공유자격증명을 파일 형태로 만들어야 한다.

운영체제	기본 위치 및 파일 이름
Linux/macOS	~/.aws/config 또는 ~/.aws/credentials
Windows	%USERPROFILE%\.aws\config 또는 %USERPROFILE%\.aws\credentials

- **.aws\credentials 파일**
  - 다운로드한 CSV 파일을 열어, Access Key ID와 Secret Access Key를 복사해서 붙여넣는다.

credentials - Windows 메모장

파일(F) 편집(E) 서식(O) 보기(V) 도움말(H)

```
[default]
region= ap-northeast-2
aws_access_key_id = AKIA5FTZAHOMYC6X4MYO
aws_secret_access_key = 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
```

---

5. AWS SDK for JavaScript

- AWS 폴더 생성
- **npm init -y**
  - 프로젝트를 초기화하기 위한 명령
  - 해당 폴더 내에 package.json 파일 생성
  - dir로 확인

```
PS C:\Windows\system32> cd c:\aws
PS C:\aws> npm init -y
Wrote to C:\aws\package.json:

{
  "name": "aws",
  "version": "1.0.0",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "description": ""
}



PS C:\aws> dir
```

```
    디렉터리: C:\aws


Mode                LastWriteTime         Length Name
----                -------------         ------ ----
-a----      2025-02-01  오후 10:17            217 package.json
```

- **npm install 명령어**
  - S3를 사용하기 위한 AWS SDK 패키지 설치

```
PS C:\aws> npm install --save @aws-sdk/client-s3

added 102 packages, and audited 103 packages in 7s

2 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

- **dir 명령어**
  - package.json 파일 확인

```
PS C:\aws> dir


    디렉터리: C:\aws


Mode                LastWriteTime         Length Name
----                -------------         ------ ----
d-----      2025-02-01  오후 10:21                node_modules
-a----      2025-02-01  오후 10:21          64067 package-lock.json
-a----      2025-02-01  오후 10:21            279 package.json
```

- **package.json 파일의 내용**

- 의존성 패키지 목록인 dependencies 항목: AWS SDK 패키지가 목록에 추가된 것 확인

package.json - Windows 메모장

파일(F) 편집(E) 서식(O) 보기(V) 도움말(H)

```
{
  "name": "aws",
  "version": "1.0.0",
  "main": "index.js",
  "scripts": {
    "test": "echo ₩"Error: no test specified₩" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "description": "",
  "dependencies": {
    "@aws-sdk/client-s3": "^3.740.0"
  }
}
```

- 마지막 줄에 “type”: “module” 추가
  - 작성할 때 주의할 점은 JSON 포맷이 깨지지 않도록 마지막 줄에 콤마를 넣은 후 작성

```
{
  "name": "aws",
  "version": "1.0.0",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "description": "",
  "dependencies": {
    "@aws-sdk/client-s3": "^3.740.0"
  },
  “type”: “module”
}
```

- index.js 파일 생성

```
import {
    S3Client,
    CreateBucketCommand,
    PutObjectCommand,
} from '@aws-sdk/client-s3';

export async function main() {
    const s3client = new S3Client({});

    // S3 버킷 생성
    const bucketname = 'mjc-bucket-12345';//자신만의 버킷 이름(미리 만들지 말 것
    await s3client.send(
        new CreateBucketCommand({
            Bucket: bucketname,
                LocationConstraint: 'ap-northeast-2',
        })
    );
    // S3 버킷에 파일 업로드
    await s3client.send(
        new PutObjectCommand({
            Bucket: bucketname,
            Key: 'hello.txt',
            Body: 'Hello, JavaScript SDK!',
        })
    );
}
main();
```

- 작성한 코드 실행

```
PS C:\aws> node index.js
```

- 만약 코드가 정상적으로 실행되었다면 AWS 콘솔의 S3 페이지에서 새로 생성된 버킷 확인
- 업로드된 hello.txt 객체 존재

- 다운로드 후에 파일을 열면 내용은 다음과 같다.

```
Hello, JavaScript SDK!
```

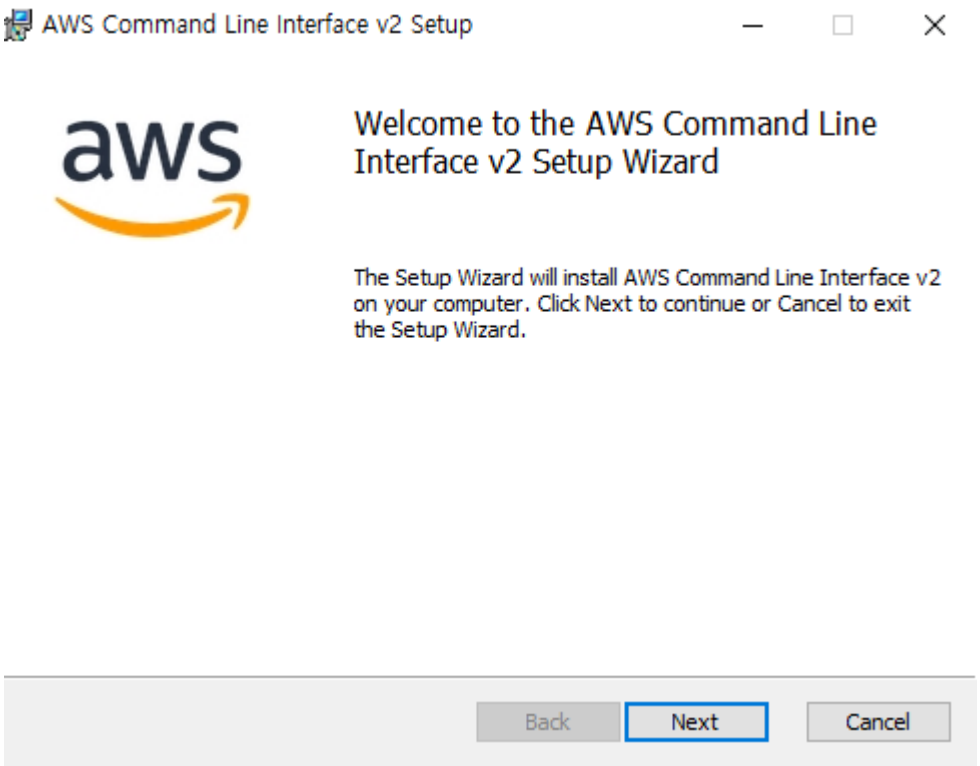
---

6. AWS CLI 설치 및 사용

- 최신 버전의 AWS CLI 설치 또는 업데이트

https://docs.aws.amazon.com/ko_kr/cli/latest/userguide/getting-started-install.html

- **설치**



- **aws --version**
  - AWS CLI 설치 확인

```
PS C:\aws> aws --version
aws-cli/2.23.11 Python/3.12.6 Windows/10 exe/AMD64
```

- **aws s3 ls**

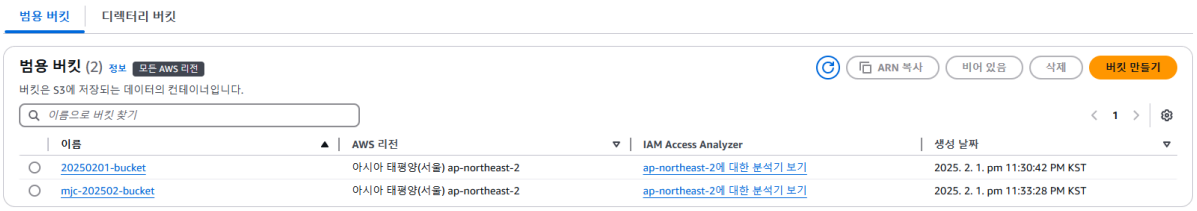
- S3 버킷 목록 출력
- 미리 콘솔에서 버킷을 생성한 후 확인

```
PS C:\aws> aws s3 ls
2025-02-01 23:30:43 20250201-bucket
```

- **aws s3 mb s3://버킷이름**
  - 버킷 생성

```
PS C:\aws> aws s3 mb s3://mjc-202502-bucket
make_bucket: mjc-202502-bucket
```

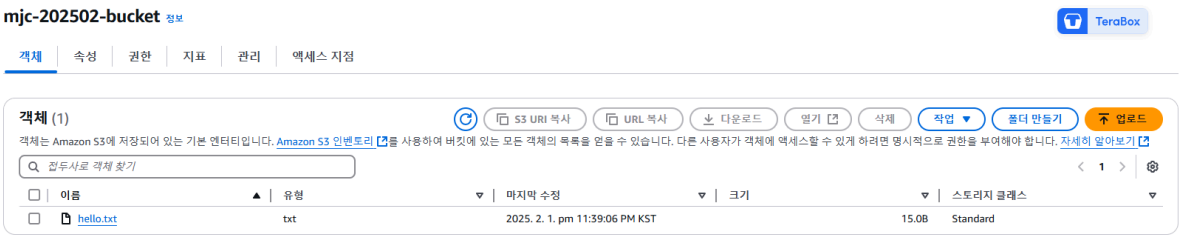
- **AWS 콘솔에서 확인**



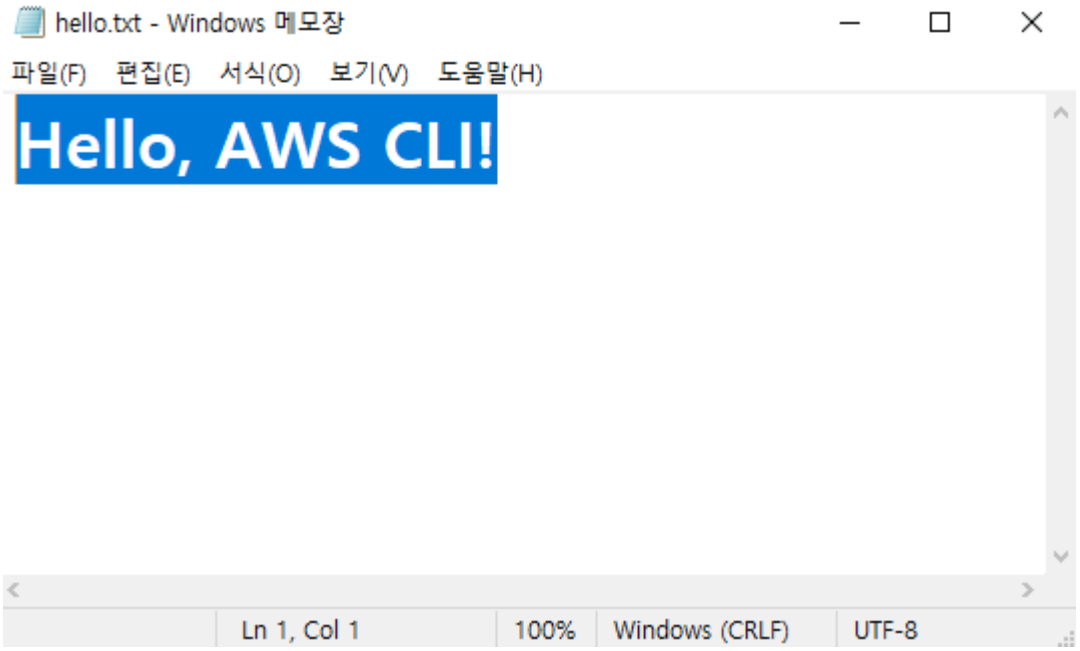
- **aws s3 cp 파일이름 s3://버킷이름**
  - 파일 업로드
  - hello.txt 파일에 “Hello, AWS CLI!” 입력 후 저장

```
PS C:\aws> aws s3 cp hello.txt s3://mjc-202502-bucket
upload: .\hello.txt to s3://mjc-202502-bucket/hello.txt
```

- **AWS 콘솔에서 확인**



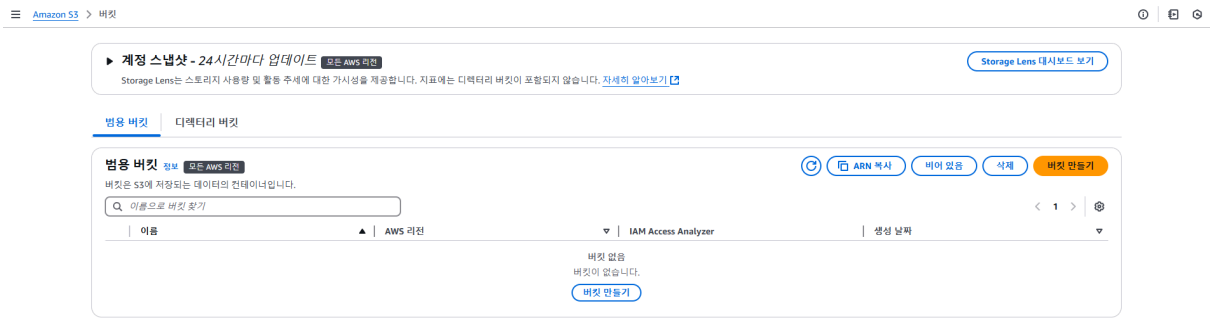
- **다운로드 후 내용 확인**



- **aws s3 rb s3://버킷이름**
  - 버킷 삭제 → 에러: 버킷이 비어있지 않기 때문

```
PS C:\aws> aws s3 rb s3://mjc-202502-bucket
remove_bucket failed: s3://mjc-202502-bucket An error occurred (BucketNot
```

- **aws s3 rb --force s3://버킷이름**
  - 내부에 있는 객체와 버킷이 모두 삭제

```
PS C:\aws> aws s3 rb --force s3://mjc-202502-bucket
delete: s3://mjc-202502-bucket/hello.txt
remove_bucket: mjc-202502-bucket
```

- AWS 콘솔에서 확인